Unified Master Systems Requirements Document (SRD)

Module: Core Inventory, Fulfillment, &Point-of-Care Consumption Ecosystem
Document Version: 6.0-Unified-Master
Target Architecture: Zero-Code Multi-Tenant Microservice Framework (Laravel Native)
System Status: Approved Architectural Direction
Audience: Backend Engineers, UI Engineers, Database Engineers, Integration Engineers

1. Core Design Objectives & Governing Principles

1.1 Structural System Mandates

The Inventory Module shall maintain absolute physical stock balances, track multi-tier inventory consumption patterns, and seamlessly support both prepaid (upfront collection) and postpaid (floor stock consumption) clinical workflows while maintaining minimal front-end user complexity.

1.2 The Two Governing Architectural Maxims

- **The Floor Principle:** Users record what happened physically at the point of care. The system determines the underlying inventory, consumption, reconciliation, and financial consequences automatically.
- **The Domain Isolation Principle:** The Inventory Module owns physical truth; the Main Module owns financial truth. The Inventory Module shall not perform internal billing or insurance calculations; it communicates only verified inventory facts (e.g., "This item was consumed"), leaving post-processing to the central financial core.

2. Global Identity & Multi-Tenant Infrastructure

The platform governs multi-site operations by running all transaction lookups against a metadata matrix controlled entirely via an administrative settings sidebar, guaranteeing zero-code multi-tenancy.

2.1 The Dual-Identity Core Framework

All database entries must validate tracking operations against a strict dual-key identity model:

- **Permanent Client ID (client_id):** A static, lifetime identifier assigned uniquely to a single patient file. This key anchors all historical charts, cumulative ledger balances, and cross-day queue inheritance models.
- **Expiring Visitor ID (visitor_id):** A dynamic operational key tracking active daily hospital

foot traffic. At exactly 00:00:01, the database resets this sequence to zero and increments it chronologically as daily entries check in. Daily dashboard views must filter active arrays exclusively using the current tracking day parameters to maximize database index scanning speeds.

2.2 Goods vs. Services State Rule

- **Services (e.g., Consultations):** Process through a three-tier state machine (Pending → In Progress → Completed) to manage physical clinic sub-lines.
- **Goods (e.g., Pharmaceuticals):** Subject to a strict binary state restriction loop within the Main Module: State Transition: [Pending] → [Completed]. Goods are structurally barred from utilizing or displaying an intermediate In Progress status or occupying physical service sub-lines within the Main Module dashboard.

2.3 The Tenant Capability Configuration Engine

The Inventory Module supports site-specific feature activation through configuration variables managed entirely via the admin settings panel. Features must be toggled on or off without requiring software source code alterations:

- Floor Stock Management
- Crash Cart Management
- Batch / Lot Tracking
- Serial Number Tracking
- Internal Ordering Frameworks
- Automated Stock Counts
- Multi-Store Network Operations

2.4 The Label Dictionary System

The system supports tenant-specific terminology substitution dynamically at the user interface layer while internal database system keys remain permanently fixed. If a tenant label override is defined in settings, the UI displays the tenant label; otherwise, the system falls back to the platform default.

Platform Internal Key	**Hospital Tenant Terminology Override**	**Retail Tenant Terminology Override**
Client	Patient	Customer
Client Space	Ward	Shop/Location
Inventory Item	Drug	Product
Store	Pharmacy/Vault	Stockroom

Platform Internal Key	Hospital Tenant Terminology Override	Retail Tenant Terminology Override
Usage Record	Charted Administration	Consumption Log

3. Inventory Network Hierarchy & Node Typology

The system supports a strict parent-child distribution network. Stock movement is highly restricted and shall only occur directly from a parent store to a child store down the relational tree: Main Store (1) → Distribution Store (2-4) → End Store (5-6) → Satellite Store (7).

3.1 Warehouse & Store Node Specifications

1. **Main Store (Root Node):** The central facility warehouse. Acts as the primary vendor receiving hub. It can only distribute stock to child nodes; it cannot execute direct consumer transactions.
2. **Distribution Store (Intermediate Node):** Sub-warehouses, sectional depots, or bulk storage vaults. Manages bulk redistribution down the tree. Blocked from executing client sales.
3. **End Store (Dispensary Gate):** Local unit pharmacies or retail checkout points. This is the exclusive operational gate authorized to execute direct client sales, issue prepaid goods, or stage floor inventory.
4. **Satellite Store (Floor Stock Points):** Physical consumption nodes positioned directly beneath an End Store parent. Examples include Ward Stock, ICU Stock, Theatre Stock, Emergency Department Stock, and Crash Carts.

3.2 Dynamic Variable-Tier Asset Locations

To support diverse facilities, the database utilizes an abstract, three-tiered spatial coordinate setup: location_layer_3 (Macro), location_layer_2 (Meso), and location_layer_1 (Micro). The layout interface maps plain-English labels to these columns dynamically based on store setup records (e.g., a central warehouse maps Aisle → Rack → Pallet, while an End Store pharmacy maps Wall → Cabinet → Bin).

4. Inbound Queue Ingestion & Forward Fulfillment Pipeline

4.1 Standard Forward Workflow Ingestion

The standard forward workflow processes chronologically from prescription to dispensing: Doctor Prescription → Translator Engine → Billable Inventory → Main Module Billing →

Payment Confirmation → Inventory Queue. When a payment confirmation registers, the item hits the Inventory Queue as an individual line item. The system avoids notification spam by executing an automated lookup check:

1. The module reads the Client Space ID linked to the incoming payment record.
2. It cross-references the Client Space Assignment Settings Registry to locate the mapped End Store ID.
3. The item is stamped with that specific End Store ID, stripping it from the global pool. It renders exclusively on the terminals of users logged into that specific pharmacy sub-node.

4.2 Fulfillment Strategy Routing

The ingestion engine routes incoming queue records based on the strategy variable assigned to the originating space:

- **Outpatient Strategy (DISCRETE_IMMEDIATE):** Line items flash instantly onto the specific counter viewport as an individual express ticket for immediate micro-picking from local dispensing bins.
- **Inpatient Strategy (BATCH_AND_STAGE):** Items accumulate quietly inside ward reservoirs, automatically grouping into Patient Baskets (consolidating multiple lines belonging to the same client session).

4.3 Urgent / STAT Prioritization Interceptor

If an inbound prescription metadata payload contains priority strings defined in administrative settings (e.g., [STAT] or [URGENT]), the item bypasses all chronological sorting. It drops to the absolute top of the active fulfillment queue, applies a distinct red background highlight (#FFECEC), and triggers a recurring 60-second alert tone until acknowledged.

4.4 Spatial Batch Picking Engine

When an inpatient ward collection run is triggered, the engine loops through the active pool and executes a multi-tier path sorting query:

1. It sums the total units required for each unique SKU across all patient baskets in that ward's reservoir.
2. It automatically sorts the collection order lines along the abstract layout coordinates defined for that store node: Sort Array: location_layer_3 → location_layer_2 → location_layer_1.
3. It prints a Sequential Picking Route Sheet, guiding the runner in a smooth, one-way path down the pharmacy shelves to eliminate backtracking.
4. **The Staging Step:** Pulled items are split into patient-specific envelopes and placed inside a barcode-labeled physical Ward Delivery Tote. The technician scans the tote, updating the queue rows to STAGED. Physical inventory is removed from shelves but is not yet written off the End Store balance sheet. Inventory shall only be reduced when dispensing is completed.

4.5 The Token-Exchange Ward Handshake Protocol

To protect data integrity and keep ward nurses out of the inventory application interface, the physical handoff of a staged tote enforces a strict token validation sequence:

1. Transitioning a tote to a staged state transmits an automated ready alert to the corresponding ward dashboard inside the Clinical Module.
2. The nurse clicks "Collect Medications" on their clinical terminal, reviews an independent itemized checklist generated from active patient charts, and receives a secure, time-locked, single-use 5-Digit Verification Code bound to that active nurse's shift session.
3. The nurse walks to the pharmacy counter and cross-checks the physical envelopes inside the tote against their clinical checklist. If verified, they give the 5-digit code to the pharmacist.
4. The pharmacist types the code into the End Store screen. The system runs an API validation call to the Clinical Module. If the token matches, the stock is released.
5. **Partial Acceptance Sub-Routine:** If an error is spotted for a specific patient package during inspection, the pharmacist flags that line. The system isolates the discrepancy, duplicates it into a new database row, clears its tote references, and rolls it back to a Pending state for urgent correction, while allowing the nurse's code to successfully clear and release the remaining correct items.

5. Point-of-Care Usage Recording & Usage Reconciliation Engine

5.1 The Patient Approved Pool Framework

The Inventory Module shall maintain a system-managed, digital Approved Pool for every patient. The Approved Pool serves purely as a clinical reconciliation mechanism and does not represent physical inventory balances. It contains item rows that have been successfully prescribed, billed, paid for, and dispensed via the forward pipeline.
[Patient Approved Pool Container]
Item Line 1: Paracetamol 30 Tablets
Item Line 2: Ceftriaxone 10 Vials
Item Line 3: IV Fluids 2 Bags

5.2 Unified Usage Recording Framework

The interface provides a unified "Record Usage" action for clinical staff on the floor. Clinical personnel record what happened; they are not required to determine billing status, inventory balances, financial configurations, or stock adjustments. The system runs these evaluations automatically via a backend reconciliation engine when entries are saved. Every usage log must be mapped to an explicit context:

- **Patient Usage:** Associated with a specific patient (e.g., Medication Administration, Patient Supplies, Patient Consumables). Required fields: Patient, Item, Quantity.
- **Administrative Usage:** Not associated with a patient (e.g., Cleaning, Disinfection, Spill Management, Training, Ward Operations). Required fields: Administrative Purpose, Item, Quantity. Purposes are configurable by tenant.

5.3 The Usage Reconciliation Engine Routing Logic

Whenever a usage entry is committed, the engine evaluates the parameters against three explicit operational scenarios:

- **Scenario A: Approved Item Usage (Prepaid Workflow)**
  • Condition: The administered item and quantity exist within the patient's digital Approved Pool.
  • System Actions: The engine reduces the Patient Approved Pool balance directly (e.g., Ceftriaxone drops from 10 → 8).
  • System Prohibitions: The system shall NOT reduce physical inventory, generate new billing lines, or notify the Main Module.
  • Reason: The physical inventory was already deducted from stock logs when the standard dispensing handoff occurred at the End Store counter.
- **Scenario B: Non-Approved Patient Usage (Postpaid Workflow)**
  • Condition: An item is consumed from a floor stock cupboard or satellite store, and it does not exist within the patient's active Approved Pool.
  • System Actions: 1. Instantly reduce the physical inventory balance of the originating Satellite Store node. 2. Commit an immutable usage tracking log. 3. Dispatch an asynchronous message payload upstream to the Main Module billing gate.
  • Main Module Execution: The Main Module receives the inventory fact, checks insurance configurations, and automatically generates post-paid billing entries, payment collection structures, or client financial records.
- **Scenario C: Administrative Usage (Operational Expenses)**
  • Condition: An item is recorded under an administrative context (e.g., cleaning materials or spill cleanups).
  • System Actions: Reduce the physical inventory balance of the satellite store, record consumption metrics, and commit an entry to the forensic audit ledger.
  • System Prohibitions: The system shall NOT generate billing records, link the action to a patient account, or notify the Main Module billing lines.

5.4 Summary Matrix of Inventory Deductions

Physical and digital inventory updates must adhere strictly to the following validation parameters:

System Context Scenario	Physical Inventory Reduced	Digital Approved Pool Reduced	Triggers Main Module Billing Notification
Dispensing Completed (End Store Counter)	Yes	No (Populates Pool)	Yes (Closes Paid Ticket)
Approved Patient Usage (Bedside MAR)	No	Yes	No
Non-Approved Floor Stock Usage	Yes	No	Yes
Administrative Usage (Ward Utility)	Yes	No	No
Crash Cart Reconciliation Usage	Yes	No	Yes

6. Emergency Crash Cart Workflow

Crash Carts must be treated as specialized, locked Satellite Store nodes within the database schema. They implement an emergency state machine that eliminates data entry barriers during active clinical crises.
READY STATUS → Action: Deploy Cart / Open Box → State Shifts to DEPLOYED
READY STATUS ← Action: Physically Restock & Reseal ← RECONCILING

- **Step 1: Deployment:** The user interacts with a rapid action trigger: Deploy Crash Cart or Crash Cart Opened. The node's state transitions instantly from READY to DEPLOYED.
- **Step 2: Emergency Management:** The user interface locks all input fields; no inventory documentation or tracking data shall be required during the active emergency.
- **Step 3: Post-Stabilization Reconciliation:** After the clinical event stabilizes, the user triggers the RECONCILING state and executes the standard Record Usage action to input items consumed during the crisis (e.g., Adrenaline, Atropine).
- **Step 4: System Processing:** When the reconciliation list saves, the module automatically handles four backend events simultaneously:
    1. Reduces the physical stock balances inside that specific Crash Cart node.
    2. Updates central consumption forecasting statistics.
    3. Dispatches a data summary packet to the Main Module for postpaid client billing processing.
    4. Auto-generates a high-priority replenishment requirement ticket to restock the cart.
- **Step 5: Reset to Ready:** Once a technician restocks the physical cart and inputs a validation seal number, the status returns to READY.

7. Internal Ordering & Moving-Average Replenishment Engine

7.1 Internal Ordering Philosophy

The internal supply chain operates entirely on a "Days of Stock" coverage model. Users are completely shielded from calculating raw item units; they shall not be required to configure item-level reorder quantities or manual safety stock numbers. The system performs all quantity calculations internally.

7.2 Store Replenishment Configuration

Every store node profile includes two unified settings parameters that apply uniformly across all items stored within that specific node:

- **Reorder Level (Days):** The minimum safety stock horizon (e.g., 5 Days).
- **Maximum Stock Level (Days):** The target fulfillment storage volume (e.g., 365 Days).

7.3 Current Stock Days Calculation Formula

The software must continuously evaluate a store's stock coverage duration using a rolling 15-Day Moving Average Consumption model:
Current Stock Days = Current Stock Quantity ÷ 15-Day Moving Average Consumption
This rule shall apply uniformly regardless of the targeted ordering horizon.

7.4 Consumption Forecast Selection Matrix

When generating replenishment orders to bring a store back up to its maximum stock days, the system selects its consumption forecasting window dynamically based on the requested stock coverage period to avoid seasonal or short-term data distortions:

Requested Coverage Horizon Window	System Forecast Data Source
Less than 15 Days	15-Day Moving Average Consumption Model
15 to 30 Days	30-Day Moving Average Consumption Model
30 to 90 Days	90-Day Moving Average Consumption Model
90 to 180 Days	180-Day Moving Average Consumption Model
180 to 365 Days	365-Day Moving Average Consumption Model

7.5 Order Generation & Flexible Editing Mechanics

- **Draft Generation:** The replenishment wizard runs the forecast selection query and outputs an internal order draft containing actual calculated unit quantities (e.g., Required Days x Moving Average Forecast = Suggested Actual Units).
- **Bidirectional Editing:** Generated internal orders must remain fully editable by both the requesting child store and the supplying parent store. Staff can toggle the editing mode between two options:
    - **Quantity Mode:** Modifying raw numeric box or vial counts directly.
    - **Days Mode:** Adjusting the targeted coverage days parameter.
- **Auto-Recalculation:** The interface must automatically recalculate raw unit quantities whenever days metrics are modified. This mechanism serves as the core resource allocation engine when a parent warehouse has insufficient stock to satisfy all child store requests completely.

8. Core Automation & Cross-Module State Sync Loops

8.1 Automated Main Module State Synchronization

To maintain a binary state machine for goods inside the Main Module without duplicate manual data entry, developers must implement a database event listener connected to local inventory milestones:

- **The Picking Trigger:** When an outpatient order is opened for review or an inpatient line item is bundled into a Ward Batch Pick List, the system leaves the Main Module status as Pending. This keeps the ticket visible in the active daily line.
- **The Completion Trigger:** The exact millisecond an outpatient checkout is confirmed or an inpatient 5-digit verification token validates at the counter, the engine dispatches an asynchronous background REST webhook upstream to the Main Module. This automatically flips the billing ticket status from Pending directly to Completed, closing out the record cleanly.

8.2 Cross-Day Active Ticket Inheritance Engine

If an outpatient pays for medications but leaves the hospital floor without collecting them, the order remains strictly Pending in the database, anchored permanently to their Client ID. At midnight, the expired Visitor ID is wiped from the active screens to maintain dashboard rendering performance.

- **The Re-Activation Hook:** When that client returns on a subsequent day, the system enforces a strict rule: they must register at the reception desk to receive a current-day active Visitor ID.
- **The Execution:** The exact millisecond their new daily profile commits, a database trigger automatically executes a query restricted by the lookback window integer configured in the sidebar settings. It pulls all historical outstanding Pending records matching that permanent Client ID, maps them to today's active Visitor ID, and drops the forgotten prescription back onto the live terminal queue seamlessly.

8.3 Fault Tolerance & Offline Queuing

If network connectivity between a local End Store node and the Main Module drops, the local inventory engine must queue all state synchronization updates (such as transitions to Completed) inside an internal encrypted cache table. A background service must continuously poll the network interface. The exact millisecond active connectivity is restored, the daemon must read the cached rows in strict chronological sequence, transmit the updates upstream to close out billing ledgers, and mark the records as processed once confirmation clears.

Addendum A-01: Enhanced Consumption Classification & Expiry Isolation

1.1 Purpose & Framework Scope

This addendum establishes a forensic classification schema for all inventory depletion events to support multi-dimensional auditing, data analytics, management reporting, cost tracking, and departmental loss control. To prevent algorithmic corruption, consumption classification is strictly a reporting and governance mechanism; it remains structurally independent of core baseline stock-forecasting and internal ordering logic.

1.2 Revised Consumption Classification Categories

Every physical depletion event must be assigned exactly one of the following hardcoded categories at the point of care:

- **PATIENT:** Inventory consumed directly for the physical treatment or care of an individual client record (e.g., Medications, IV Fluids, Clinical Supplies).
- **ADMINISTRATIVE:** Inventory consumed for standard operating tasks not attributable to an individual patient account (e.g., Cleaning, Disinfection, Training Activities, General Departmental Usage).
- **CRASH_CART:** Inventory consumed during emergency procedures originating from a specialized, locked Crash Cart node (e.g., Adrenaline, Atropine, Resuscitation Materials).
- **WASTAGE_OPERATIONAL:** Materials lost or rendered unrecoverable during routine operations due to immediate clinical contamination, broken vials during a shift, or damaged seals.
- **WASTAGE_EXPIRED:** Material assets that pass their strict manufacturer validation or expiration dates on the shelf without being assigned to a clinical transaction.

1.3 The Replenishment Independence & Expiry Exclusion Rule

The inventory module treats consumption categories equally for replenishment purposes with one vital exception:

- **The Invariance Parameter:** PATIENT, ADMINISTRATIVE, CRASH_CART, and

WASTAGE_OPERATIONAL depletions contribute fully to inventory consumption totals without weighting adjustments. This ensures busy wards automatically replace broken items to maintain daily capacity.

- **The Expiry Exclusion:** WASTAGE_EXPIRED events are **completely excluded** from moving averages, current stock days, reorder thresholds, and internal order calculations. Expiry is a metric of over-stocking or poor shelf rotation, not true clinical demand. Including it would cause an artificial over-ordering loop.
- **Physical Expiry & Stock Take Protocol:** Expired stock must be physically pulled from active dispensing bins and dropped into a locked "Expired Escrow Zone." The inventory state drops to zero to prevent dispensation, and the assets rest in this escrow zone until documented and written off during physical stock-taking audits.

1.4 Multi-Dimensional Reporting Requirements

The system shall maintain separate reporting metrics for each consumption category, allowing management to generate distribution profiles summarized across five distinct pillars:
Dimensions: [Store] x [Department] x [Inventory Item] x [Client Space] x [Date Range]

Addendum A-02: Hardcoded Parallel Demand Tracking & Dual-Stream Replenishment Engine

2.1 Core Architectural Mandate

Demand Tracking is a hardcoded, structural pillar of the core Inventory Module framework. It cannot be deactivated or masked by any tenant capability profiles. The module must maintain two parallel, independent 24-hour historical datasets inside the inventory database:

- **The Consumption Stream:** Logs verified physical depletions from shelves (Prepaid checkouts + Ward Floor Stock consumption + Administrative usage + Operational wastage).
- **The Demand Stream:** Logs pure clinical/user intent captured at the point of origin (via the Translator Engine), documenting every single product or material item requested by a user, completely blind to current shelf counts or commercial billing status.

2.2 Ingress Interception Mechanics

The system captures true unfulfilled demand by placing an immutable ledger hook inside the ingestion workflow:

1. When an item is ordered, the payload passes to the item lookup engine before stock validation occurs.
2. The engine maps the request to a master catalog SKU and immediately saves the entry to the inventory_demand_ledgers table, tracking the requested volume even if the current physical stock count is zero.
3. If an item has been entirely out of stock for months, consumption balances register as 0, but the parallel demand snapshots capture the true unfulfilled deficit volume.

2.3 The Dual-Mode Procurement Interface

Internal Requisition Forms (Child-to-Parent store links) and External Purchase Order Forms (Store-to-Vendor lines) must expose an explicit front-end toggle control switch to allow procurement staff to alter the engine's core forecasting equations dynamically:
Forecasting Basis Selector: [CONSUMPTION | DEMAND]

- **In Consumption Mode:** Moving average daily velocities are computed strictly using physical depletion snapshot records.
- **In Demand Mode:** The replenishment engine ignores shelf tracking limitations and bases moving averages entirely on historical user intent logs, allowing procurement to calculate exactly how much stock is required to completely clear an active facility deficit in a single ordering cycle.

Addendum A-03: Hardcoded Core Audit Trail & Forensic Logging Framework

3.1 Core Architectural Mandate

The Audit Trail Engine is a hardcoded, structural pillar of the core application framework. It runs automatically in the background, cannot be disabled or bypassed by any tenant profile configurations, and writes to an immutable database ledger.

3.2 The Forensic Logging Schema

Whenever an inventory state changes—including counter dispensing, bedside administration, administrative usage, crash cart reconciliation, or operational/expiry wastage—the system must capture an exact point-in-time snapshot. The ledger must record the following parameters inside a single atomic database write:

- **The Actor:** The unique identifier (user_id) of the authenticated staff member executing the action.
- **The Temporal Key:** A high-precision timestamp tracking the exact second the database transaction committed.
- **The Operational Context:** The system event path triggering the adjustment (e.g., DISPENSE, PATIENT_USAGE, ADMIN_OPEX, WASTAGE_OPERATIONAL, WASTAGE_EXPIRED).
- **The Delta Matrix:** The precise material change recorded, mapping the Old State Quantity against the New State Quantity.
- **The Local Identifier Keys:** The exact Store ID, Client ID (if applicable), and Inventory SKU tied to the event.

3.3 Integrity & Anti-Tampering Constraints

- **Immutability:** The audit ledger table is structurally restricted to INSERT actions only. Application database roles are stripped of UPDATE, DELETE, or DROP permissions on

this table.

- **Sequential Verification:** Every log entry must generate a SHA-256 hash that links it chronologically to the hash of the row preceding it. Any manual database tampering or row deletion instantly breaks the hash chain, immediately alerting system administrators during standard automated health checks.